KOTLIN USER GROUP BERLIN

One Topic to Rule 'Em All

Taming domain-event order with Kafka & Avro

Yonatan Karp-Rudin

30 min talk + 10 min Q&A

github.com/yonatankarp/one-topic-to-rule-em-all

CHARACTER SHEET

Adventurer: Yonatan Karp-Rudin

Class: Staff Engineer · **Guild:** Billie (Berlin)

Weapons of choice: Kotlin, Spring Boot, and event-driven systems that occasionally fight back

Yonatan

Inventory: `ff4k` · `kotlin-design-patterns` · `kotlin-spring-boot-template`

Current quest: *making domain events arrive in the order they happened*

`github.com/yonatankarp` · `yonatankarp.com`

3 A.M. - YOUR PAGER

Something just leveled up a hero who doesn't exist

> 💥 `LEVEL-UP FOR UNKNOWN ADVENTURER ad-7f3c1 - the ledger has never heard of them!`

Every event was delivered. Exactly once. In the order you produced them.

Every CRUD shop has this bug - wearing a different costume.

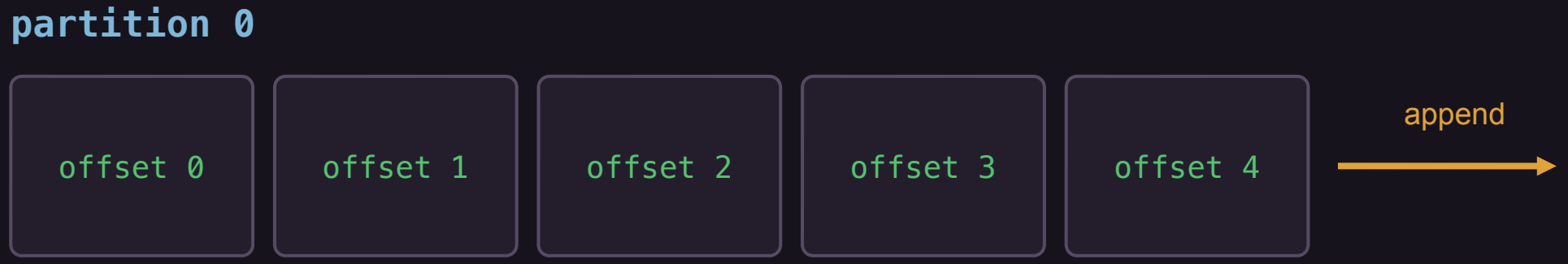
THE SETUP EVERYONE BUILDS

One topic per event type. Looks clean. Ships fast.

Type-named topics. Clear ownership. Easy diagram. Approved in review.

WHAT KAFKA ACTUALLY GUARANTEES

Ordering exists within a partition. Full stop.



Producer writes in order → consumer reads in that exact order.

The partition is the unit of order. Nothing larger is ordered.

COROLLARY

Across topics, there is NO ordering. None.

`guild.adventurer_registered`

lag: high (busy consumer)

`guild.adventurer_leveled_up`

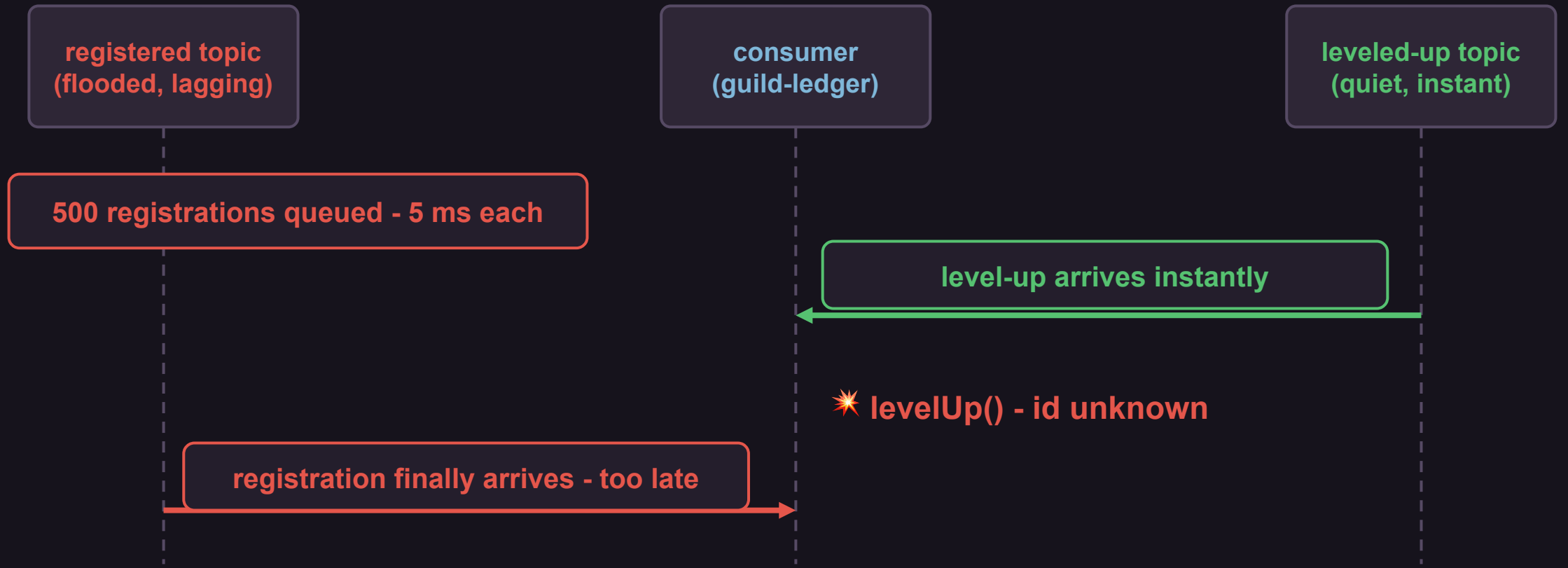
lag: ~zero (idle consumer)

Two topics = two independent clocks. They drift.

Star Wars shipped IV-VI before I-III. Your topics do this daily.

THE RACE IN SLOW MOTION

How the level-up overtakes the registration



THE TEMPTING NON-FIX

"Can't I just buffer and reorder by timestamp?"

- How long do you wait for the maybe-late event?
- Whose clock? Producer skew, NTP drift, GC pauses.
- What if it never comes - is it lost, or just slow?
- **You now own a stateful reordering buffer and its bugs.**

You've rebuilt a partition. Badly.

THE FIX

All events of one aggregate go to one topic, keyed by the aggregate id.

Aggregate: the thing whose events must stay in order.

Same key → same partition → guaranteed order. That's the whole trick.

BUT THE REGISTRY SAYS NO

Register a 2nd schema on guild.adventurers-value

```
POST /subjects/guild.adventurers-value/versions
  { schema: AdventurerLeveledUp }

409 Conflict
Schema being registered is incompatible with
an earlier schema for subject
"guild.adventurers-value"
```

Default subject naming assumes ONE schema per topic. A 2nd type looks like a breaking change.

THE MENTAL-MODEL FIX

A subject is not a topic

- Topic - Kafka's storage & ordering unit (the log).
- Subject - the registry's versioned compatibility scope.
- **Compatibility is checked per subject - not per topic.**
- **topic ↔ schema as 1:1 is just the DEFAULT naming strategy.**

Change the naming strategy and one topic can carry many independently-versioned schemas.

THE THREE SUBJECT-NAMING STRATEGIES

Pick the subject pattern, change everything

TopicNameStrategy

`<topic>-value`

`guild.adventurer_registered-value`

The default - and the cage.

RecordNameStrategy

`<record FQN>`

`org.guild.adventurer.AdventurerRegistered`

Schema roams any topic; niche.

TopicRecordNameStrategy

`<topic>-<record FQN>`

`guild.adventurers-org.guild.adventurer.AdventurerRegistered`

Many schemas, scoped per topic.

WINNER

THE ONE LINE

application-fixed.yml

```yaml
spring:
  kafka:
    producer:
      properties:
        # The one line this talk is about:
        value.subject.name.strategy:
          io.confluent.kafka.serializers.subject.TopicRecordNameStrategy
```

Producer-side only. Consumers read the schema id from the record bytes - no consumer config change.

THE SCHEMA

Avro IDL - AdventurerRegistered.avdl

```
@namespace("org.guild.adventurer")
protocol AdventurerRegisteredProtocol {
  /** A new adventurer signs the guild ledger. High-volume event. */
  record AdventurerRegistered {
    string adventurer_id;
    string name;
    string character_class;
    timestamp_ms registered_at;
  }
}
```

DEMO CAST - THE ADVENTURERS' GUILD

Four lifecycle events. One hero.

`AdventurerRegistered`	*the flood - 500/run*
`AdventurerLeveledUp`	*rare - the overtaker*
`QuestAccepted`	*occasional*
`AdventurerDied`	*the finale*

Our hero: **Bob the Brave.** *Find the bug before my consumer does.*

MEET THE HERO

Bob the Brave

Class: Level 1 Fighter

Quest line:

`Registered` → `LeveledUp` → `QuestAccepted` → `Died` *(rocks fell)*

This is who we're rooting for. Watch his lifecycle stay in order - or not.

◆ LIVE DEMO

Act 1

Two topics. Watch the level-up overtake.

→ to the terminals

WHAT YOU JUST SAW

The overtake, and the receipts


```
✸ LEVEL-UP FOR UNKNOWN ADVENTURER ad-7f3c1 — the ledger
  has never heard of them!
▸ QuestAccepted has nowhere to go in this topology
```

Not a Kafka bug. Kafka did exactly what we asked: two topics, two clocks.

THE SPRING PAYOFF

The class reads like the aggregate's lifecycle

```
@KafkaListener(topics = ["guild.adventurers"], groupId = "guild-ledger")
class GuildLedgerListener(/* ... */) {
  @KafkaHandler fun on(event: AdventurerRegistered) { ... }
  @KafkaHandler fun on(event: AdventurerLeveledUp)  { ... }
  @KafkaHandler fun on(event: QuestAccepted)        { ... }
  @KafkaHandler fun on(event: AdventurerDied)       { ... }
  @KafkaHandler(isDefault = true)
  fun unknown(event: Any) {
    log.warn("Unhandled event type: {}", event::class.simpleName)
  }
}
```

◆ LIVE DEMO

Act 2

One topic. One line changed. Order holds.

→ to the terminals

CHOOSE YOUR DISPATCH

@KafkaHandler vs. when - same hole

@KafkaHandler - framework dispatch

- One method per type.
- Spring routes by deserialized type.
- Forgotten type → silent isDefault, at runtime.

when dispatch - one explicit place

- All routing in one greppable when.
- Avro classes aren't sealed → else required.
- Forgotten type → falls to else, at runtime.

Neither gives compile-time safety on raw Avro types. For that, map events into your own sealed hierarchy at the boundary.

Both variants live in the repo - profiles `fixed` / `fixed-when`.

Name things so the topology stays obvious

- Topics: domain.aggregate (e.g. guild.adventurers).
- Event types: fully-qualified record names (org.guild.adventurer.*).
- **One subject per (topic, type) under TopicRecordNameStrategy.**
- **Version only on a breaking change - not per release.**

The topic name is the consistency boundary. Make the boundary readable.

PRODUCTION NOTES II - GOVERNANCE

Let CI own the registry, not your producers

- Schemas live in a central repo, reviewed like code.
- CI registers schemas and gates compatibility per (topic, type) subject.
- **Producers run with auto.register.schemas = false.**
- **A breaking change fails the pipeline - not 3 a.m. in prod.**

Compatibility is a build-time gate, not a runtime surprise.

TAKEAWAYS

Four things to take home

1. Ordering is a partition guarantee - and nothing larger.
2. One aggregate → one topic, key = aggregate id.
3. TopicRecordNameStrategy is the knob that unlocks it.
4. @KafkaHandler makes the multi-type topic pleasant.

github.com/yonatankarp/one-topic-to-rule-em-all

FIND ME

github.com/yonatankarp · linkedin.com/in/yonatankarp
medium.com/@yonatankarp · yonatankarp.com

APPENDIX

Backup slides

Evolution · union schemas · consumer cost · when type-per-topic is right

BACKUP · B1 - SCHEMA EVOLUTION

Evolve one event without touching its siblings

- Compatibility is checked per subject = per (topic, event type).
- Add a field to AdventurerRegistered → only that subject evolves.
- **AdventurerLeveledUp, QuestAccepted, Died are untouched.**
- **Independent version histories per type on the same topic.**

BACKUP · B2 - THE ALTERNATIVE

Union schema + schema references

- Model the topic value as one union of all event types (via schema refs).
- Single subject, single schema id namespace for the topic.
- Trade-off: atomic evolution of the whole topic shape…
- **…but you lose per-type compatibility isolation.**

TopicRecordNameStrategy = many subjects, isolated. Union = one subject, atomic. Pick your coupling.

Every consumer sees every type on the topic

- All consumers deserialize all types landing on the topic.
- Filtering is consumer-side (e.g. @KafkaHandler / when).
- **Fine for aggregate streams - types are few and related.**
- **Wrong for firehose topics - many unrelated, high-volume types.**

Topic-per-event-type is correct when…

- The streams are genuinely independent - no shared aggregate.
- No ordering relationship to protect between the events.
- Heterogeneous retention or scaling per type.
- **No single aggregate id ties the records together.**

The rule isn't 'one topic always'. It's: order within an aggregate → one topic.